

0.68. Voir la position du A en direct

Le 0.67, plus deux lignes dans la boucle : `nombre(0, 17, x)` et `nombre(4, 17, y)`. La position du A s'affiche à chaque image, pendant qu'on le déplace.

1. Ce qu'il faut comprendre

C'est le 0.67, plus deux lignes dans la boucle : `nombre(0, 17, x);` et `nombre(4, 17, y);`.

Ce qui est nouveau ici : `nombre()` **DANS la boucle.** Au 0.20, `nombre()` était appelé une fois, après un trajet : il montrait la valeur à ce moment-là. Ici, il est dans `while (true)` : il est rappelé à **chaque image**, 60 fois par seconde. L'écran montre donc toujours la valeur **du moment** : c'est de l'affichage **en temps réel**.

Ce qu'on voit : en bas à gauche, trois chiffres pour `x` (la colonne du A), et à côté, trois chiffres pour `y` (sa ligne). Appuie sur les flèches : les nombres changent en même temps que la lettre bouge. Au départ, `009` et `008`, le milieu de l'écran.

Pourquoi avant `poser` : si le A va sur la ligne 17, il passe **par-dessus** les nombres, parce qu'il est posé après eux. Les nombres sont réécrits à l'image suivante, dès que le A repart.

Sans rien écrire, la page le montre aussi : sous la console, ouvre «  Inspecteur », onglet **Variables** : `x`, `y` et `attente` y sont, par leur nom, mis à jour 4 fois par seconde.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1622 octets** de cartouche.

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// x, y, attente : voir l'onglet variables.h. (#include veut être seul sur sa ligne.)
#include "variables.h"
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    while (true) {
        image();                                // attend l'image suivante

        if (attente > 0) attente--;

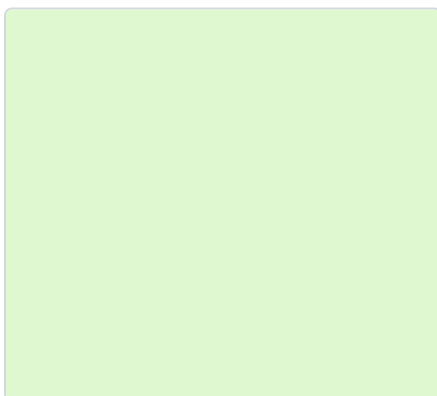
        if (attente == 0) {
            effacer(x, y, 1);                    // efface l'ancienne place
            if (bouton(DROITE) && x < 19) { x++; attente = 8; }
            if (bouton(GAUCHE) && x > 0) { x--; attente = 8; }
            if (bouton(BAS) && y < 17) { y++; attente = 8; }
            if (bouton(HAUT) && y > 0) { y--; attente = 8; }
        }

        // Les deux lignes ajoutées : la position du A, réécrite à CHAQUE image.
        //
        //  nombre(0, 17, x);      nombre(4, 17, y);
        //      | | |              | | |
        //      | |  +-- x : la colonne | |  +-- y : la ligne du A
        //      | +----- ligne 17 : tout en bas de l'écran
        //      +----- colonne 0 et colonne 4 : côte à côte
        //
        //  En bas de l'écran : 009 008  ← le A au départ, en (9, 8)
        //                      012 010  ← après 3 pas à droite et 2 en bas
        nombre(0, 17, x);          // la colonne, en direct
        nombre(4, 17, y);          // la ligne, en direct

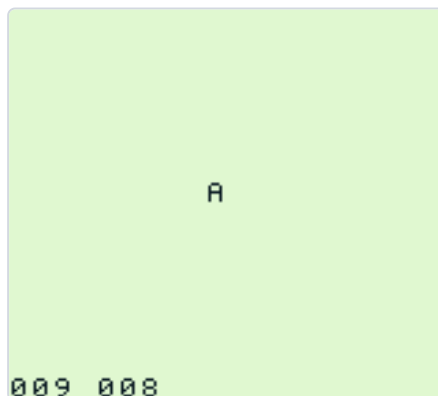
        poser(x, y, ALPHABET[0]);      // la lettre, par-dessus tout
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



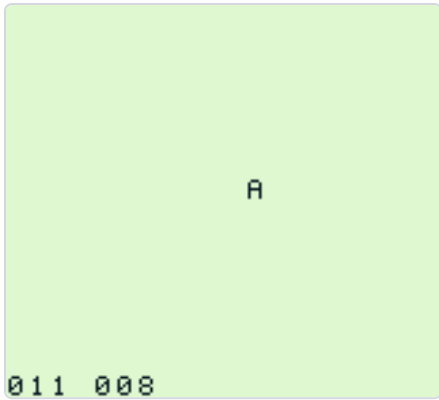
la toute première image — la console vient de démarrer — image 1



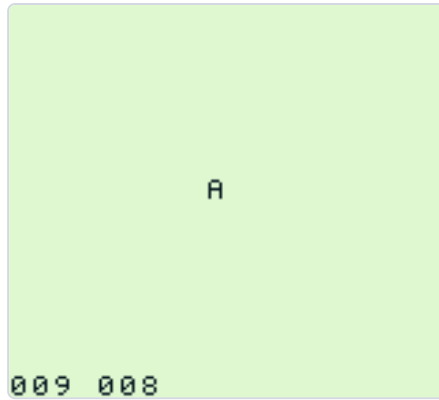
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

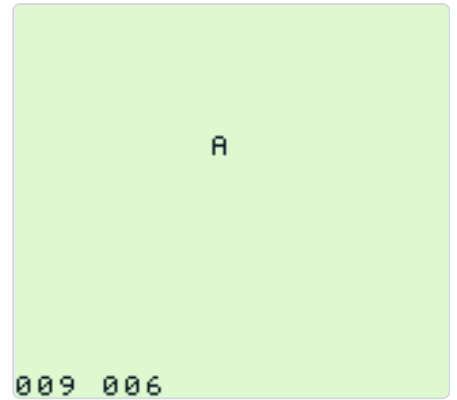
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



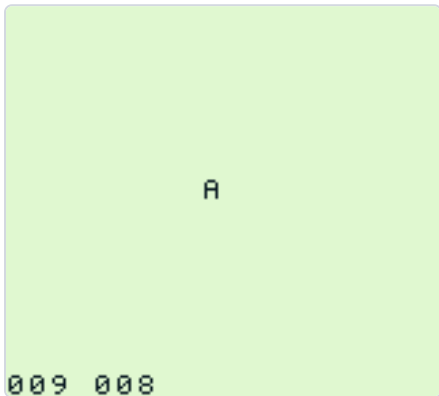
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

Le A qui se déplace avec les flèches ; en bas à gauche, sa colonne et sa ligne changent en même temps.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ au départ, 009 et 008 (« 009 008 »)

✓ le A a bougé ($x = 13$, $y = 5$)

✓ l'écran montre sa colonne (« 013 »)

✓ et sa ligne (« 005 »)

Ensuite : **0.68.1. Voir la position du A en direct — de base, ailleurs** — niveau 0 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++